

Sistema de Reservas de Restaurante

Fase 3 — Modelado UML

Aprendices:

LUIS EDUARDO BUITRAGO PASCUAS C.C. 1077228906

LAURA DANIELA AVILES PERDOMO C.C. 1007681669

ANDRES CAMILO PEREZ MENDOZA C.C. 1003895622

Instructor:

**KAROL DANIELA CORREA TRUJILLO
ING. SISTEMAS**



**SERVICIO NACIONAL DE APRENDIZAJE
CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS.
ANALISIS Y DESARROLLO DE SOFTWARE. (3239137)
NEIVA
2026**

1. Introducción	4
2. Análisis de Consistencia entre las Fases 1 y 2	5
2.1 Dirección de la dependencia entre HU-08 y HU-09	5
2.2 y 2.3 Estimación de puntos de historia de HU-06 y HU-11	5
2.4 Dependencias de HU-07 y HU-12	5
3. Objetivo de la Modelación UML	6
4. Diagrama de Casos de Uso	7
4.1 Vista del actor Cliente.....	7
4.2 Vista del actor Recepcionista.....	8
4.3 Vista del actor Administrador.....	9
4.4 Descripción	9
Relaciones <<include>> utilizadas	9
Relación <<extend>> utilizada.....	10
5. Diagrama de Clases.....	11
5.1 Descripción	11
Relaciones clave.....	11
5.2 Pilares de POO y principios SOLID aplicados	12
6. Diagrama de Secuencia — Crear una Reserva.....	14
6.1 Descripción	14
7. Diagramas de Actividades.....	15
7.1 Crear reserva.....	16
7.2 Iniciar sesión.....	17
7.3 Registrarse	18

7.4 Recuperar / cambiar contraseña	19
7.5 Consultar disponibilidad	20
7.6 Modificar reserva	21
7.7 Cancelar reserva.....	22
7.8 Gestionar mesas	23
7.9 Gestionar usuarios.....	24
7.10 Gestionar clientes.....	25
7.11 Generar reportes.....	26
7.12 Enviar / gestionar notificaciones.....	27
8. Diagrama de Estados — Clase Reserva	28
8.1 Descripción	28
9. Diagrama de Componentes	29
9.1 Responsabilidades de cada componente	29
10. Diagrama de Despliegue	30
12. Conclusiones	33

1. Introducción

Este documento corresponde a la Fase 3 del proyecto “Sistema de Reservas de Restaurante”, desarrollado como trabajo integrador del programa Análisis y Desarrollo de Software del SENA. Tomando como base conceptual la Fase 1 (historias de usuario, épicas, actores y MVP) y la Fase 2 (requerimientos funcionales, no funcionales, reglas de negocio, restricciones y trazabilidad HU-RF), esta fase representa mediante diagramas UML la estructura, el comportamiento y el funcionamiento del sistema.

El modelado se limita estrictamente al alcance definido en las fases anteriores: gestión de clientes, mesas, disponibilidad, reservas, autenticación, roles y permisos. No se incorporan funcionalidades adicionales como pagos, domicilios, inventario, facturación o gestión de cocina, por no estar contempladas en el alcance del proyecto (restricción RT-05).

2. Análisis de Consistencia entre las Fases 1 y 2

Antes de construir los diagramas definitivos se revisaron en detalle las Fases 1 y 2 en busca de contradicciones. Se identificaron cuatro inconsistencias puntuales, documentadas a continuación junto con la solución adoptada. Ninguna de ellas afecta el alcance funcional del sistema; corresponden a diferencias de detalle en dependencias o en la estimación de historias.

2.1 Dirección de la dependencia entre HU-08 y HU-09

- Fase 1 (sección 7): HU-08 depende de HU-09, argumentando que para restringir acciones por rol primero debe existir una sesión autenticada.
- Fase 2 (sección 8): HU-09 depende de HU-08, argumentando lo contrario: que iniciar sesión requiere ya conocer el rol del usuario.

Esta relación no puede ser válida en ambos sentidos porque generaría una dependencia circular imposible de secuenciar. Se adopta la dirección de la Fase 1 (HU-09 antes que HU-08): el sistema primero autentica al usuario (RF-15) para conocer su identidad y, solo después, aplica las restricciones y permisos asociados a su rol (RF-13/RF-14). Esta decisión es coherente con RN-13, que exige una sesión válida como condición para el acceso.

2.2 y 2.3 Estimación de puntos de historia de HU-06 y HU-11

HU-06 pasa de 8 puntos (Fase 1) a 13 puntos (Fase 2), y HU-11 pasa de 3 puntos (Fase 1) a 2 puntos (Fase 2). En ambos casos se adopta el valor de la Fase 2, por tratarse de una re-estimación posterior, realizada ya con los requerimientos funcionales y no funcionales definidos (en el caso de HU-06, en particular RF-09, RF-10 y RNF-04, que exigen validación segura y concurrente en base de datos).

2.4 Dependencias de HU-07 y HU-12

La Fase 1 registra dependencias más amplias que la Fase 2: HU-07 depende de HU-04 y HU-06 en la Fase 1, pero solo de HU-04 en la Fase 2; HU-12 depende de HU-01, HU-08 y HU-09 en la Fase 1, pero solo de HU-01 y HU-08 en la Fase 2. Se adopta el conjunto más completo (Fase 1), ya que las dependencias omitidas siguen siendo necesarias: modificar una reserva (HU-07) debe volver a

aplicar las reglas de no-cruce de HU-06, y la búsqueda de clientes (HU-12) solo tiene sentido dentro de una sesión autenticada (HU-09), en línea con RN-13.

Con estas cuatro decisiones documentadas, el modelo UML de esta Fase 3 se construye sobre una base conceptual única y sin contradicciones internas.

3. Objetivo de la Modelación UML

Representar, mediante diagramas UML estándar, la estructura estática y el comportamiento dinámico del Sistema de Reservas de Restaurante, manteniendo coherencia total con los actores, historias de usuario, requerimientos funcionales y no funcionales, reglas de negocio y alcance del MVP establecidos en las Fases 1 y 2. Se pone especial énfasis en representar el mecanismo que evita reservas duplicadas y el overbooking (RN-01, RF-09, RF-10), por ser la preocupación central del proyecto.

4. Diagrama de Casos de Uso

Para que cada relación se distinga con claridad, el diagrama de casos de uso se presenta en tres vistas, una por actor, en lugar de una única vista con los tres actores superpuestos. Las tres vistas comparten el mismo límite de sistema (“Sistema de Reservas de Restaurante”) y son consistentes entre sí: los casos de uso Iniciar sesión y Cerrar sesión se repiten en las tres porque son compartidos por los tres roles.

4.1 Vista del actor Cliente

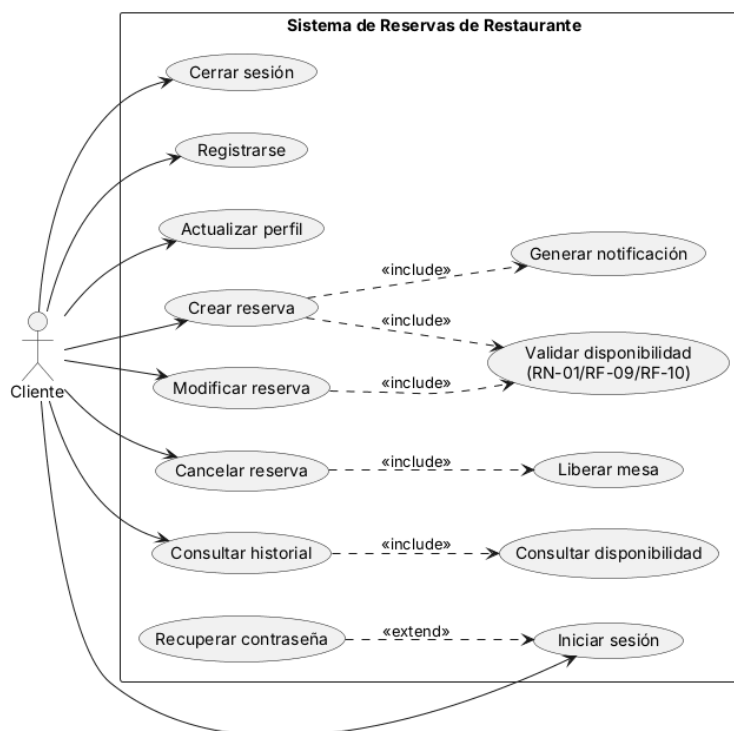


Figura 1a. Diagrama de casos de uso — actor Cliente.

4.2 Vista del actor Recepcionista

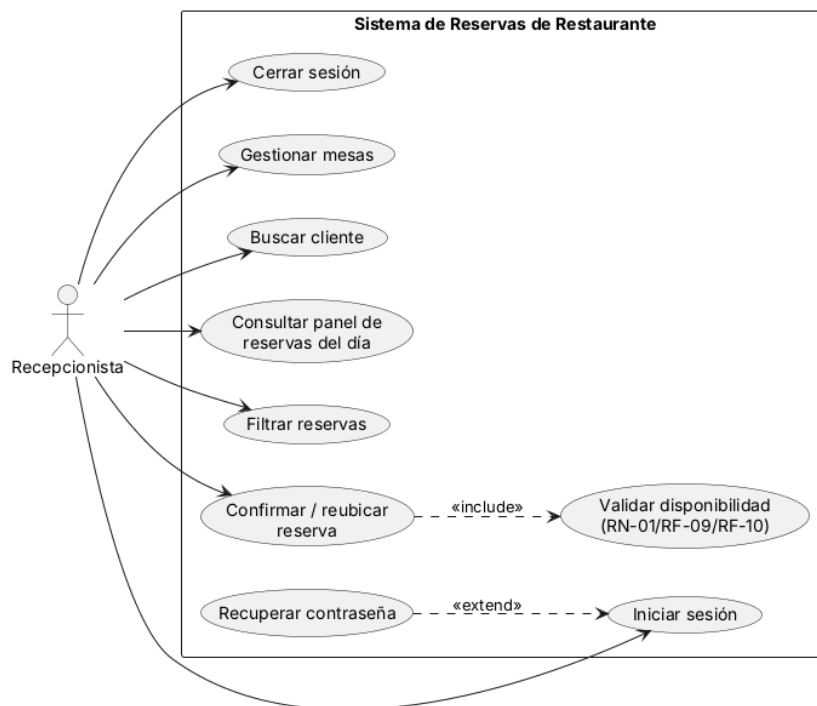


Figura 1b. Diagrama de casos de uso — actor Recepcionista.

4.3 Vista del actor Administrador

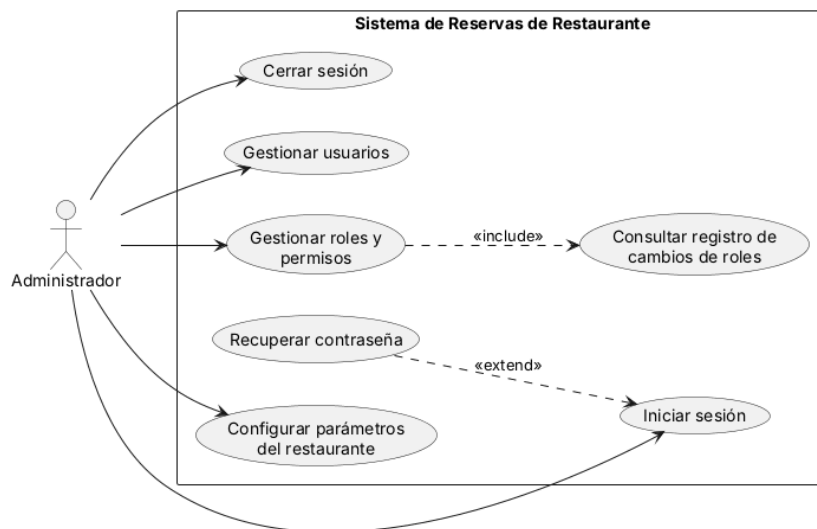


Figura 1c. Diagrama de casos de uso — actor Administrador.

4.4 Descripción

Las tres vistas cubren, en conjunto, los tres actores definidos en la Fase 1 (Cliente, Recepcionista, Administrador) y todos sus casos de uso. Iniciar sesión y Cerrar sesión aparecen en las tres vistas porque los tres roles los usan (RF-15, RF-16); se optó por repetirlos en lugar de introducir un actor abstracto “Usuario”, precisamente para que cada línea de asociación se lea sin ambigüedad dentro de una sola vista.

Relaciones <<include>> utilizadas

- Crear reserva incluye Consultar disponibilidad: no es posible seleccionar una mesa sin antes conocer cuáles están libres (RF-03 → RF-04).
- Crear reserva incluye Validar disponibilidad: toda creación exige la verificación segura de cruces de horario descrita en RN-01/RF-09/RF-10.
- Crear reserva incluye Generar notificación: cada reserva creada exitosamente notifica al cliente de forma obligatoria (RF-06).
- Modificar reserva incluye Validar disponibilidad: toda modificación de fecha, hora o personas repite la misma validación que una reserva nueva (RN-09, RF-12).

- Confirmar/reubicar reserva incluye Validar disponibilidad: el recepcionista también está sujeto a la regla de no-cruce al reubicar una reserva.
- Cancelar reserva incluye Liberar mesa: cancelar siempre libera la mesa para ese horario (RN-07).
- Gestionar roles y permisos incluye Consultar registro de cambios: cada cambio de rol/permiso queda registrado obligatoriamente (RN-12, RF-14).

Relación <<extend>> utilizada

- Recuperar contraseña extiende Iniciar sesión: es un camino opcional que el usuario toma solo si no recuerda sus credenciales (RF-20), no una parte obligatoria del inicio de sesión. Esta relación se repite igual en las tres vistas (Figuras 1a, 1b y 1c), porque la recuperación de acceso no es un privilegio exclusivo del Cliente: cualquier actor que inicia sesión (Cliente, Recepcionista o Administrador) puede olvidar su contraseña.

Entre las tres vistas se cubren las 12 historias de usuario de la Fase 1: HU-01 (Registrarse), HU-02 (Gestionar mesas), HU-03 (Consultar disponibilidad), HU-04 (Crear reserva), HU-05 (Panel de reservas del día / Filtrar reservas), HU-06 (Validar disponibilidad), HU-07 (Modificar/Cancelar reserva), HU-08 (Gestionar roles y permisos), HU-09 (Iniciar/Cerrar sesión), HU-10 (Actualizar perfil), HU-11 (Consultar historial) y HU-12 (Buscar cliente).

Nota de alcance — Configurar parámetros del restaurante: este caso de uso del actor Administrador (Figura 1c) no corresponde a ninguna historia de usuario ni requerimiento funcional formal de las Fases 1 y 2; no existe una HU ni una RF que lo respalden explícitamente. Se deriva únicamente de la descripción general del actor Administrador en la Fase 1 (sección 3), donde se menciona que este “configura los parámetros generales del restaurante (horarios, número de mesas)”. Al igual que “Generar reportes” (sección 7.11), se documenta como una extensión razonable del rol de Administrador, sujeta a aprobación como cambio del proyecto (RT-05), y no como una funcionalidad ya formalizada en el MVP.

5. Diagrama de Clases

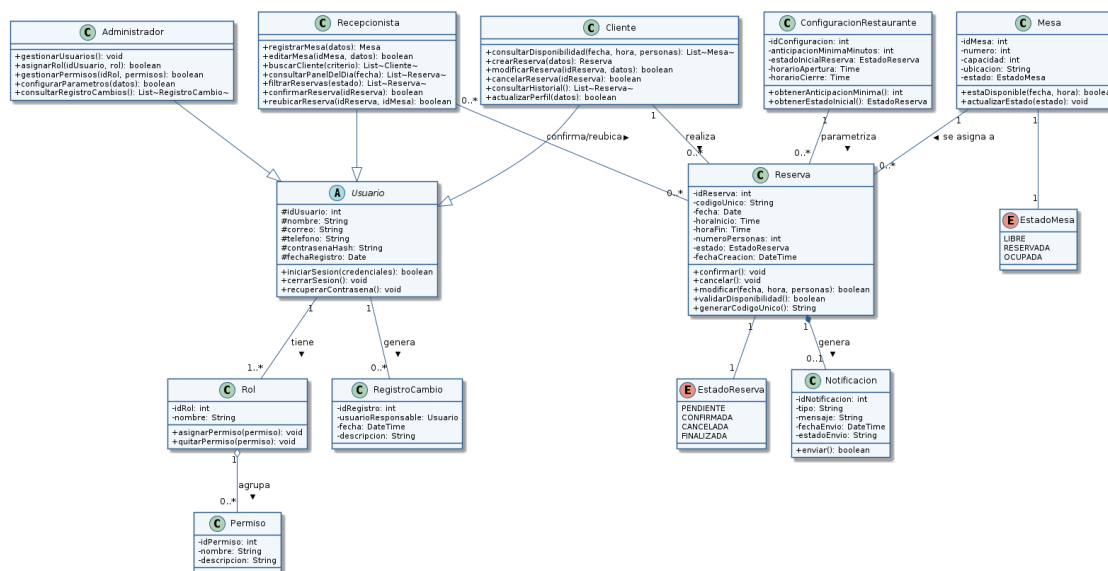


Figura 2. Diagrama de clases del Sistema de Reservas de Restaurante.

5.1 Descripción

La clase abstracta Usuario concentra los atributos y métodos comunes de autenticación (RF-15, RF-16, RF-20). Cliente, Recepcionista y Administrador heredan de Usuario y agregan los métodos propios de sus responsabilidades, evitando duplicar atributos como correo, teléfono o contraseña.

Relaciones clave

- Cliente → Reserva (1 a 0..*): un cliente puede realizar muchas reservas a lo largo del tiempo; cada reserva pertenece a un único cliente. Es la base del historial de reservas (HU-11).
- Mesa → Reserva (1 a 0..*): una mesa puede tener muchas reservas a lo largo del tiempo, pero el diagrama de secuencia y las reglas de negocio garantizan que nunca existan dos reservas activas simultáneas sobre la misma mesa y horario (RN-01).
- Usuario → Rol (1 a 1..*) y Rol → Permiso (1 a 0..*, agregación): permiten representar RF-13 (gestión de roles) y RF-14 (permisos asociados a cada rol) sin necesidad de una clase adicional para cada combinación usuario-función.
- Reserva → Notificación (composición 1 a 0..1): la notificación depende por completo de la existencia de la reserva que la origina (RF-06); si la reserva se elimina, su notificación no tiene sentido de forma independiente.

- `ConfiguracionRestaurante` → `Reserva` (1 a 0..*): centraliza parámetros como la anticipación mínima (RN-05) y el estado inicial (confirmada o pendiente, RN-06), evitando repetir esta configuración en cada reserva.

Nota de alcance — clase `ConfiguracionRestaurante`: los atributos `horarioApertura`, `horarioCierre` y `anticipacionMinimaMinutos` no están formalizados en ninguna HU ni RF de las Fases 1 y 2. Únicamente RN-05 exige respetar “una anticipación mínima configurada” y RN-06 exige un “estado inicial según la configuración del restaurante”, sin definir un mecanismo ni una interfaz para configurarlos. Esta clase se incluye porque el modelo de clases necesita representar de dónde provienen esos valores, y se apoya en la misma mención general del actor `Administrador` de la Fase 1 (sección 3) usada para el caso de uso “Configurar parámetros del restaurante”. Se documenta, por tanto, como la misma extensión razonable señalada en la sección 4.4, sujeta a aprobación como cambio del proyecto (RT-05).

- `RegistroCambio` asociado a `Usuario`: materializa RN-12, guardando quién hizo el cambio de rol/permiso, la fecha y la hora, sin necesidad de sobrecargar la clase `Rol`. Los atributos `codigoUnico` y `estado de Reserva`, junto con el método `validarDisponibilidad()`, son los elementos que permiten entender cómo se evita la duplicación de reservas: toda creación o modificación pasa por este método antes de persistir el cambio, tal como se detalla en el diagrama de secuencia.

No se crearon clases adicionales para conceptos que ya están cubiertos por atributos o enumeraciones (por ejemplo, `EstadoMesa` y `EstadoReserva` se modelan como enumeraciones y no como clases, ya que no tienen comportamiento ni atributos propios), en cumplimiento de la indicación de evitar clases innecesarias.

5.2 Pilares de POO y principios SOLID aplicados

El diagrama de clases de la sección 5 es un modelo de dominio puro (no mezcla capas de arquitectura como controladores o servicios), lo que permite evaluar directamente la aplicación de los pilares de la programación orientada a objetos:

- Abstracción: la clase abstracta Usuario define el contrato común de autenticación sin asumir un rol concreto; Reserva expone validarDisponibilidad() como la abstracción de la regla de negocio RN-01, ocultando su implementación.
- Encapsulamiento: todos los atributos son protegidos (#) o privados (-) y solo se exponen mediante métodos públicos (por ejemplo, generarCodigoUnico() en lugar de permitir asignar codigoUnico directamente).
- Herencia: Cliente, Recepcionista y Administrador heredan de Usuario y reutilizan sus atributos y métodos de autenticación, evitando duplicación de código.
- Polimorfismo: evidenciado de forma parcial. Las tres subclases de Usuario no sobrescriben ningún método heredado con comportamiento propio en el diagrama actual; si se requiere evidenciarlo de forma explícita, se recomienda declarar un método como obtenerPermisosPorDefecto() en Usuario y sobrescribirlo en cada subclase.

En cuanto a los principios SOLID, el diagrama evidencia claramente algunos y deja otros como mejoras pendientes, sin pretender forzar su aplicación donde no corresponde:

- Responsabilidad única (SRP): cada clase tiene una única razón de cambio — Usuario gestiona identidad y sesión, Reserva gestiona el ciclo de vida de la reserva, Notificacion gestiona el envío de mensajes y RegistroCambio gestiona la auditoría.
- Sustitución de Liskov (LSP): Cliente, Recepcionista y Administrador pueden usarse en cualquier lugar donde se espera un Usuario sin alterar el comportamiento esperado (por ejemplo, autenticar()).
- Abierto/Cerrado (OCP): parcialmente evidenciado por la herencia (un nuevo tipo de usuario podría agregarse extendiendo Usuario sin modificarlo), aunque el diagrama no introduce una interfaz o clase abstracta para las reglas de validación que permita extenderlas sin modificar Reserva.
- Segregación de interfaces (ISP) e Inversión de dependencias (DIP): no están representados porque el diagrama no define interfaces; Reserva depende directamente de su propia lógica de validación en lugar de una abstracción como IValidadorDisponibilidad. Esto es razonable para el nivel de un modelo de dominio de esta fase, pero se deja como mejora explícita para una fase de diseño más detallada si el instructor lo solicita.

6. Diagrama de Secuencia — Crear una Reserva

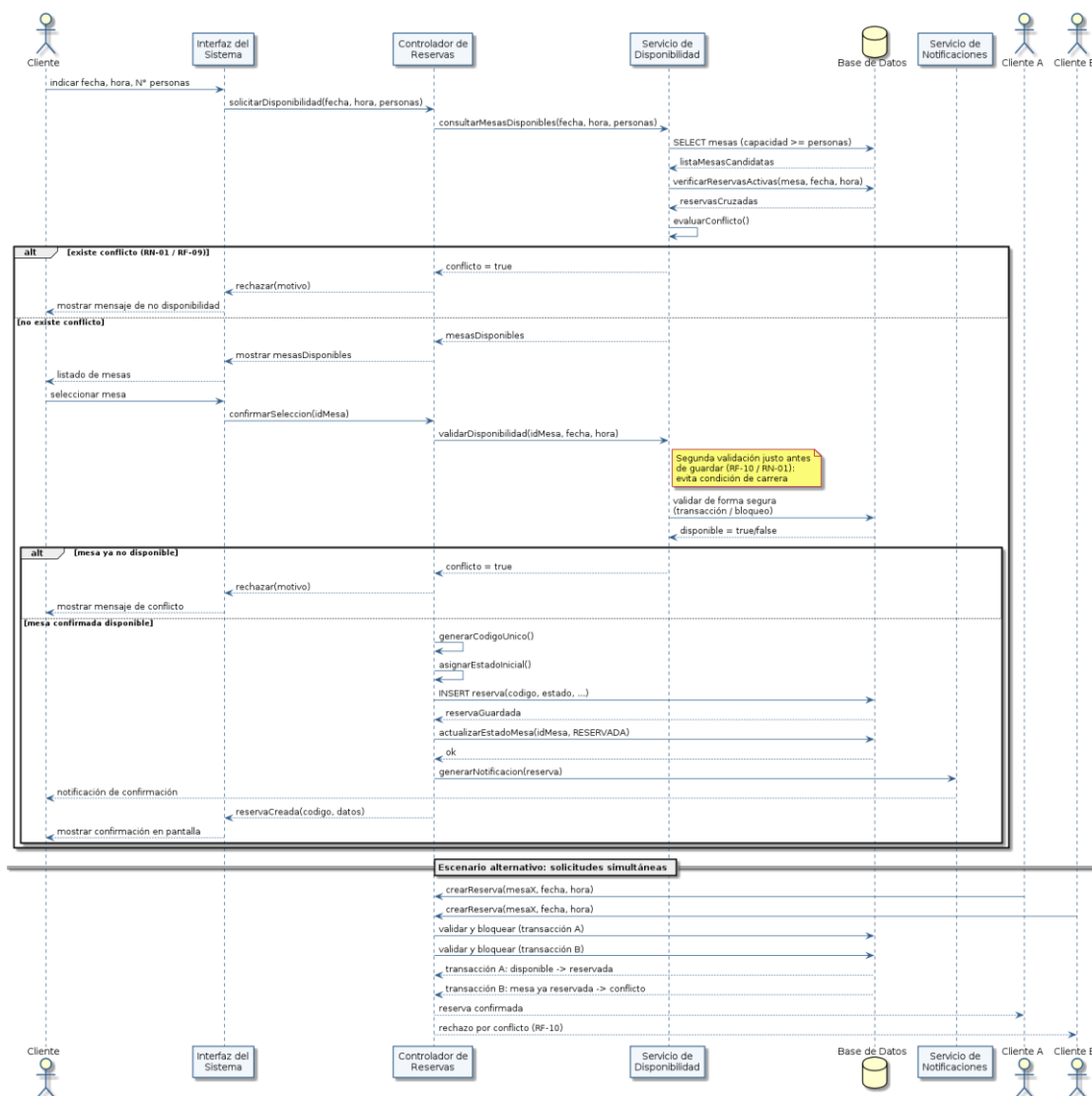


Figura 3. Diagrama de secuencia del proceso “Crear reserva”, incluyendo el escenario de solicitudes simultáneas.

6.1 Descripción

El diagrama muestra la interacción entre el Cliente, la Interfaz del Sistema, el Controlador de Reservas, el Servicio de Disponibilidad, la Base de Datos y el Servicio de Notificaciones para el proceso principal de creación de una reserva.

- El cliente indica fecha, hora y número de personas; el sistema consulta las mesas con capacidad suficiente y descarta las que tengan reservas activas cruzadas (RF-03, RN-02).
- Un fragmento alt representa la bifurcación: si existe conflicto, se rechaza la operación informando el motivo (RF-09); si no, el cliente elige una mesa.
- Antes de guardar, el Controlador vuelve a pedir al Servicio de Disponibilidad una validación final contra la Base de Datos de forma segura (transacción/bloqueo), tal como exige RF-10, para prevenir condiciones de carrera.
- Si la segunda validación falla, se rechaza; si es exitosa, se genera el código único (RF-05), se asigna el estado inicial (RN-06), se actualiza el estado de la mesa y se genera la notificación (RF-06).
- El fragmento final (separado con ==) representa el escenario alternativo exigido: dos clientes intentan reservar la misma mesa y horario al mismo tiempo; gracias a la validación segura en base de datos, solo una transacción queda confirmada y la otra es rechazada por conflicto, cumpliendo RF-10 y RNF-04.

Todos los participantes de este diagrama corresponden a clases o componentes ya definidos: el Controlador de Reservas orquesta los métodos de la clase Reserva, el Servicio de Disponibilidad implementa estaDisponible() de Mesa y validarDisponibilidad() de Reserva, y el Servicio de Notificaciones ejecuta enviar() de Notificacion.

7. Diagramas de Actividades

Se elabora un diagrama de actividades por cada proceso principal del sistema, además del correspondiente a “Crear reserva”. Todos comparten la misma notación (inicio, actividades, decisiones y fin) y son coherentes con los requerimientos funcionales, las reglas de negocio y las clases definidas en las secciones anteriores.

7.1 Crear reserva

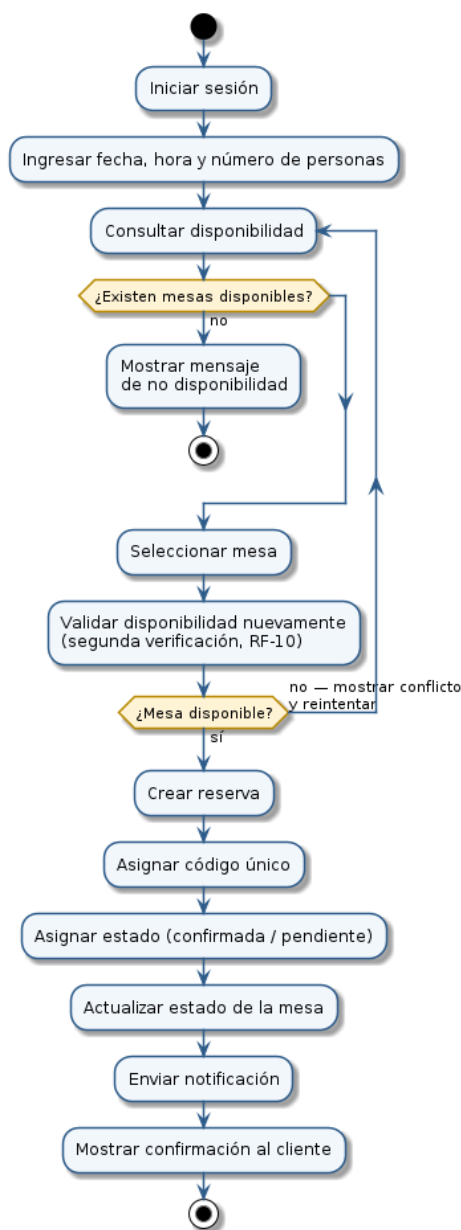


Figura 4a. Diagrama de actividades — Crear reserva.

Comienza con el inicio de sesión y el ingreso de los datos de la reserva, y utiliza dos nodos de decisión: si no existen mesas disponibles el flujo termina (RF-03); si la mesa elegida deja de estar disponible en la segunda verificación, el flujo regresa a la consulta de disponibilidad en lugar de

terminar (RF-10, RN-01). Las actividades finales (crear reserva, asignar código único, asignar estado, actualizar estado de la mesa, enviar notificación y mostrar confirmación) coinciden con los mensajes finales del diagrama de secuencia.

7.2 Iniciar sesión

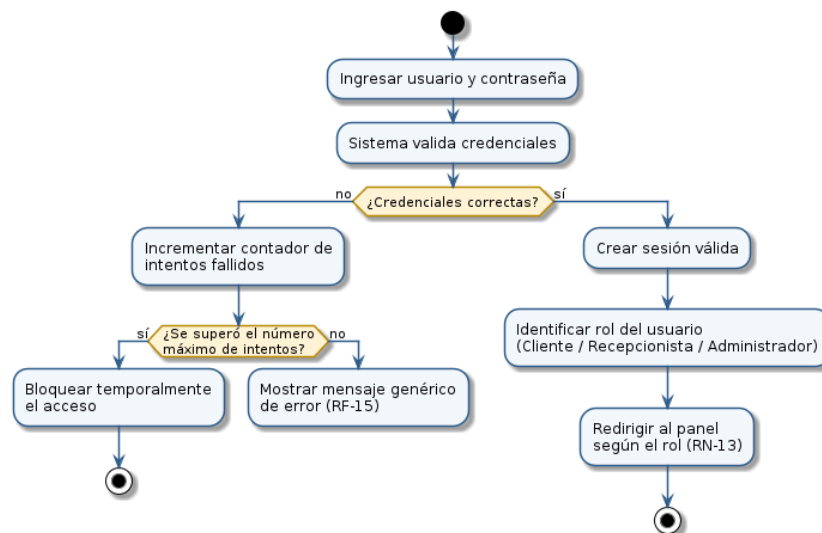


Figura 4b. Diagrama de actividades — Iniciar sesión.

Representa RF-15 y RN-13: si las credenciales son incorrectas se incrementa el contador de intentos fallidos y, al superar el límite, se bloquea temporalmente el acceso (HU-09). Si son correctas, se crea la sesión, se identifica el rol y se redirige al panel correspondiente.

7.3 Registrarse

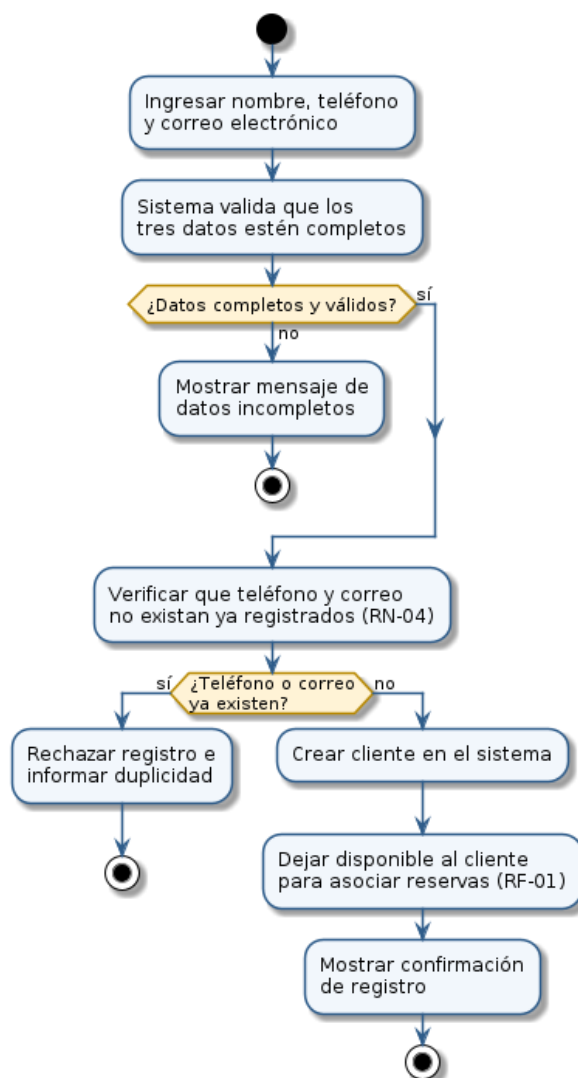


Figura 4c. Diagrama de actividades — Registrarse.

Materializa HU-01/RF-01: valida que nombre, teléfono y correo estén completos y, antes de crear el cliente, verifica que el teléfono y el correo no dupliquen un registro existente (RN-04).

7.4 Recuperar / cambiar contraseña

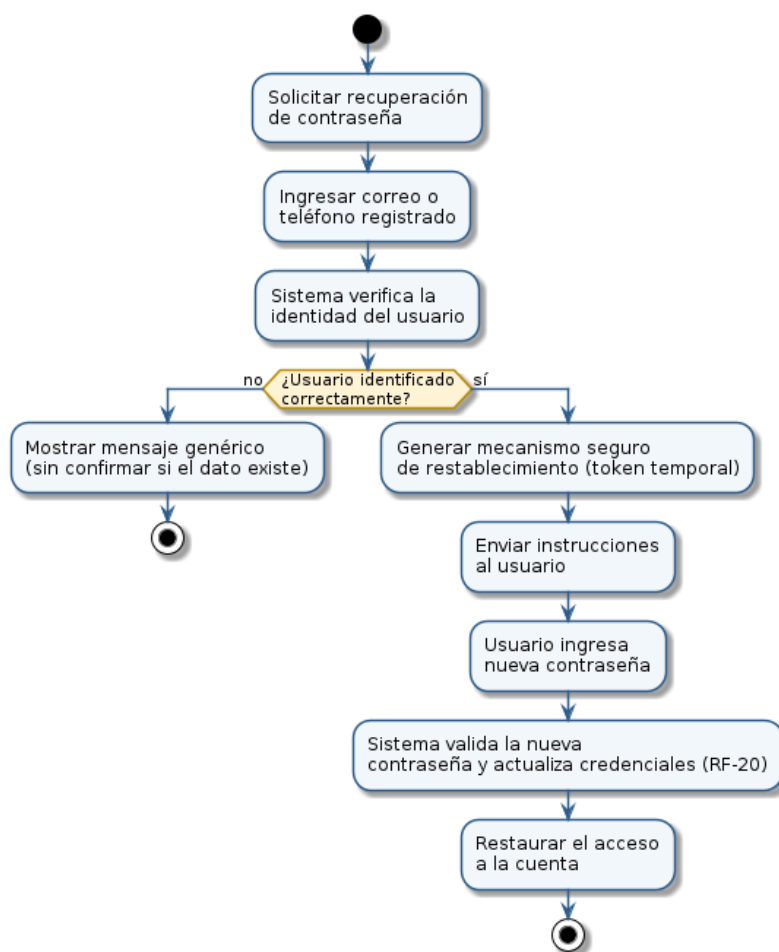


Figura 4d. Diagrama de actividades — Recuperar/cambiar contraseña.

Corresponde a RF-20: el sistema verifica la identidad del usuario sin confirmar explícitamente si el dato existe (por seguridad), genera un mecanismo temporal de restablecimiento, y solo tras validar la nueva contraseña se restaura el acceso a la cuenta.

7.5 Consultar disponibilidad

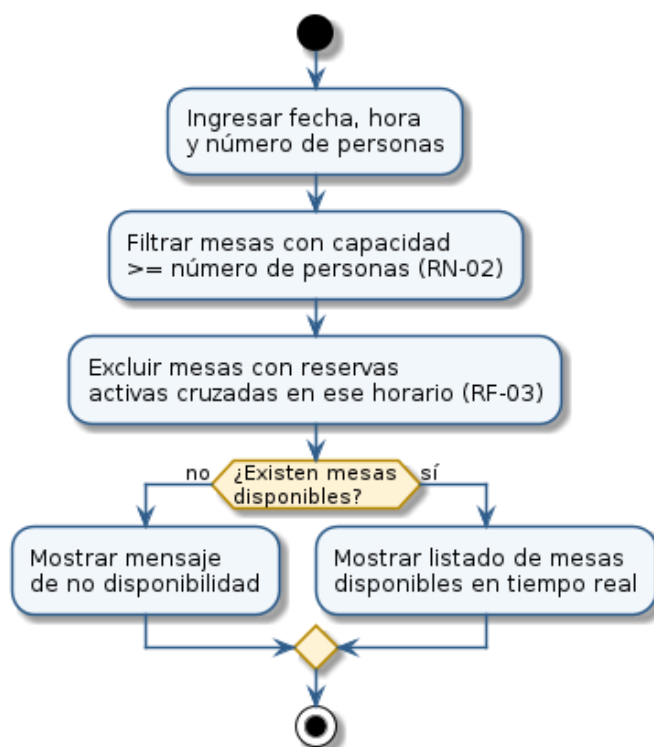


Figura 4e. Diagrama de actividades — Consultar disponibilidad.

Refleja RF-03 y RN-02: primero se filtran las mesas por capacidad suficiente y luego se excluyen las que tengan reservas activas cruzadas con el horario consultado, mostrando el resultado en tiempo real.

7.6 Modificar reserva

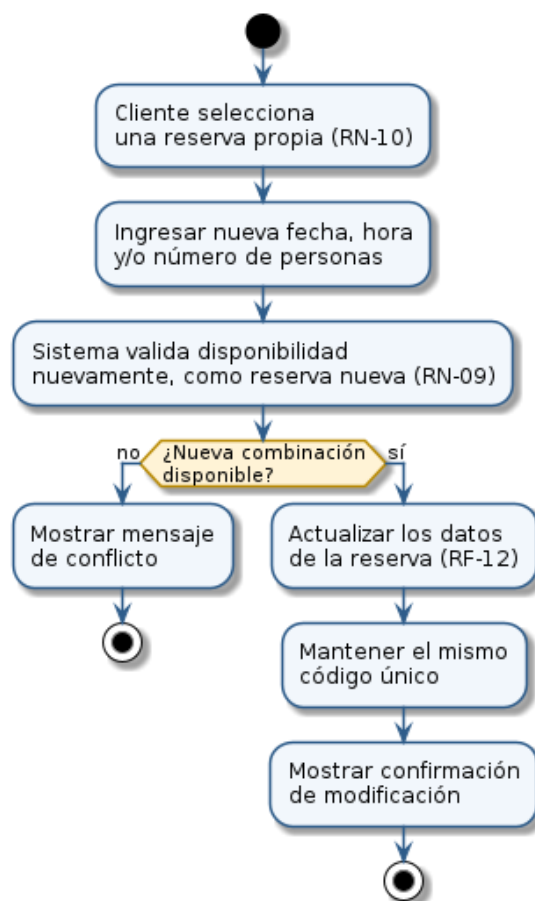


Figura 4f. Diagrama de actividades — Modificar reserva.

Aplica RN-09 y RN-10: el cliente solo puede modificar sus propias reservas, y toda modificación de fecha, hora o número de personas repite la misma validación de disponibilidad que una reserva nueva (RF-12), conservando el código único.

7.7 Cancelar reserva



Figura 4g. Diagrama de actividades — Cancelar reserva.

Representa RF-11, RN-07 y RN-08: al confirmar la cancelación, la reserva pasa a estado Cancelada, se libera la mesa para ese horario, y queda explícito que la reserva no podrá reactivarse directamente, coherente con el diagrama de estados.

7.8 Gestionar mesas

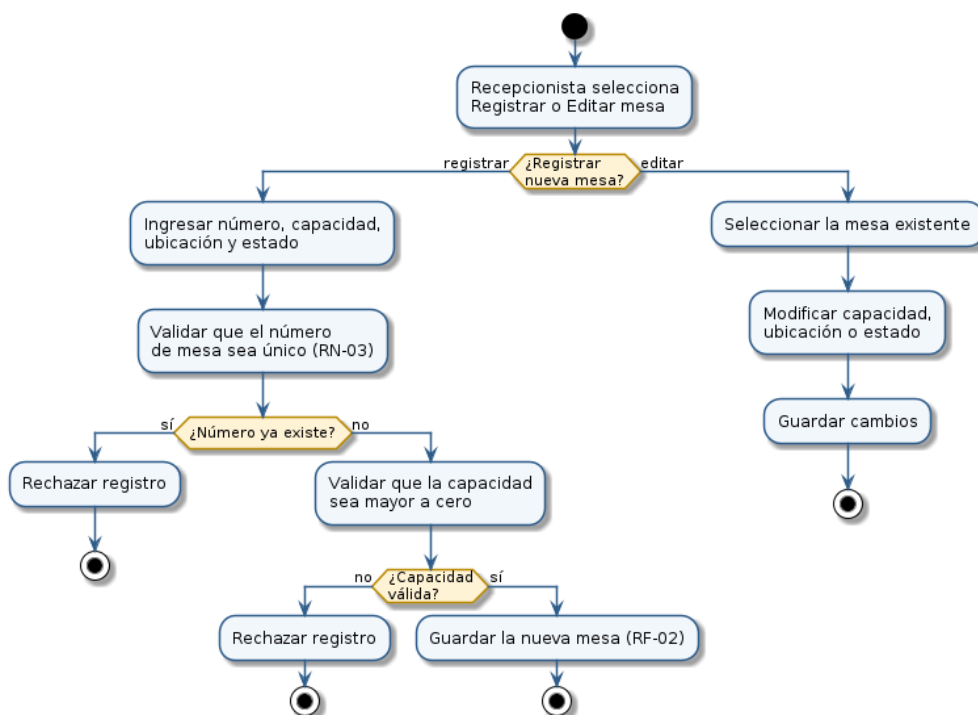


Figura 4h. Diagrama de actividades — Gestionar mesas.

Cubre RF-02: el recepcionista puede registrar una mesa nueva, validando primero que el número no esté duplicado (RN-03) y luego que la capacidad sea mayor a cero; si cualquiera de las dos validaciones falla, el registro se rechaza explícitamente. También puede editar una mesa existente.

7.9 Gestionar usuarios

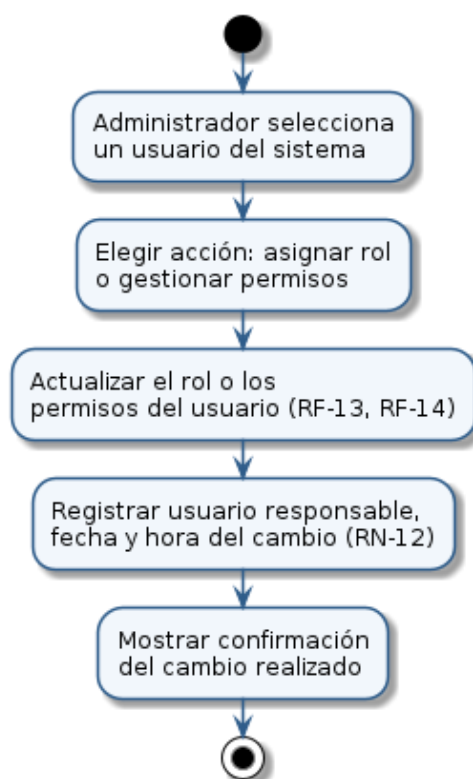


Figura 4i. Diagrama de actividades — Gestionar usuarios.

Corresponde a RF-13 y RF-14: el administrador asigna roles o permisos a un usuario, y todo cambio queda registrado con el usuario responsable, la fecha y la hora, en cumplimiento de RN-12.

7.10 Gestionar clientes

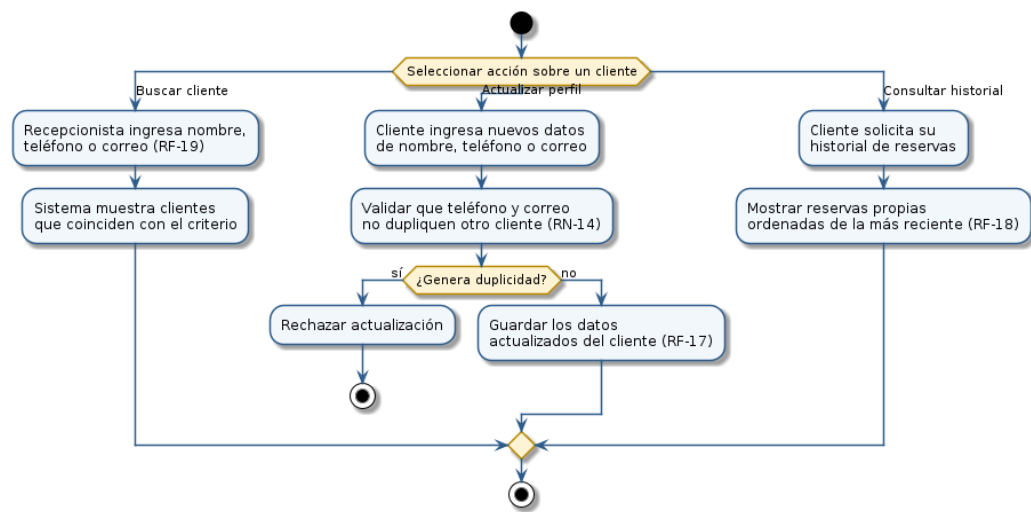


Figura 4j. Diagrama de actividades — Gestionar clientes.

Agrupar tres funciones sobre clientes que ya existían como historias independientes: buscar cliente (RF-19, HU-12), actualizar perfil validando no-duplicidad de teléfono y correo (RF-17, RN-14, HU-10) y consultar historial de reservas propias (RF-18, HU-11).

7.11 Generar reportes

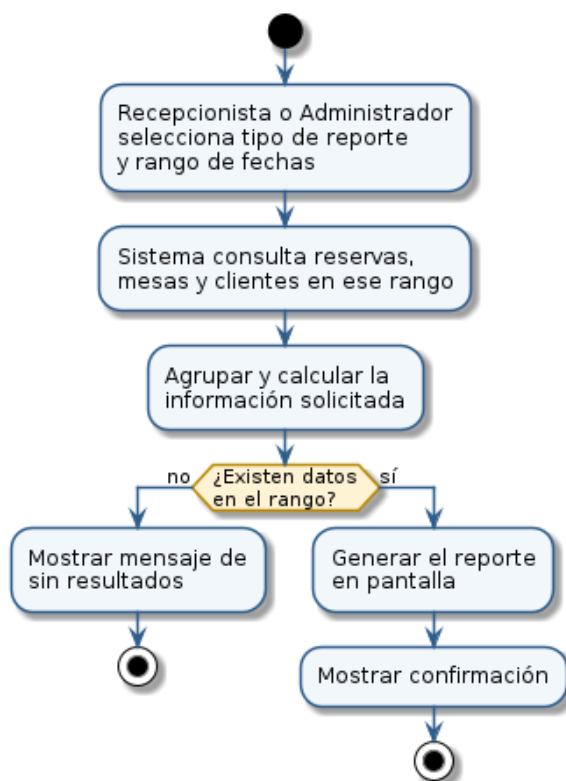


Figura 4k. Diagrama de actividades — Generar reportes.

Nota importante: la generación de reportes no corresponde a ninguna historia de usuario ni requerimiento funcional definido en las Fases 1 y 2; no existe una RF-21 ni una HU asociada. Este diagrama se incluye porque fue solicitado explícitamente para esta Fase 3, y se modela como una extensión razonable de capacidades ya existentes (el panel de reservas del día y el historial de reservas), pero debe entenderse como una funcionalidad fuera del alcance original del MVP, sujeta a aprobación como cambio del proyecto (ver restricción RT-05).

7.12 Enviar / gestionar notificaciones

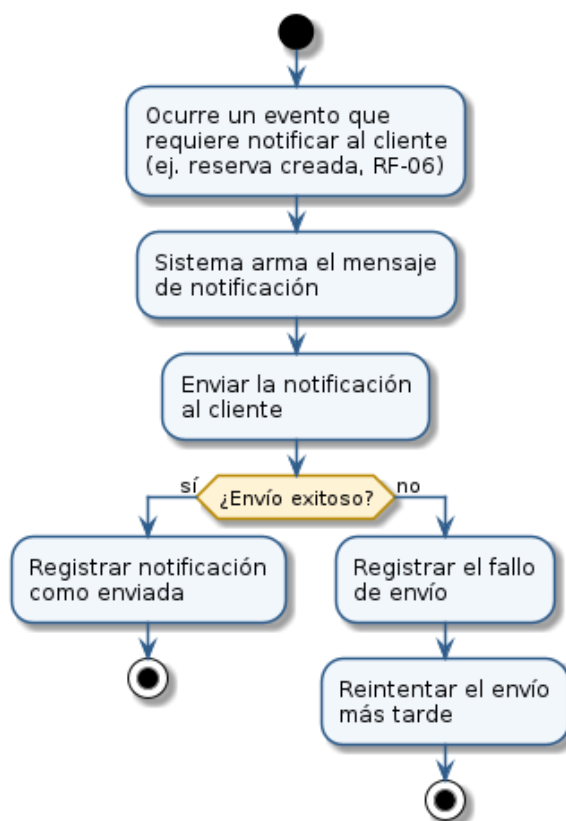


Figura 4l. Diagrama de actividades — Enviar/gestionar notificaciones.

Detalla RF-06: ante un evento que lo requiera (por ejemplo, la creación de una reserva), el sistema arma y envía el mensaje; si el envío falla, se registra el fallo y se programa un reintento, en lugar de perder la notificación.

8. Diagrama de Estados — Clase Reserva

Se incluye este diagrama porque el ciclo de vida de una reserva es el elemento más sensible del sistema frente a las reglas de negocio, y un diagrama de estados permite visualizar de forma explícita las transiciones permitidas y prohibidas.

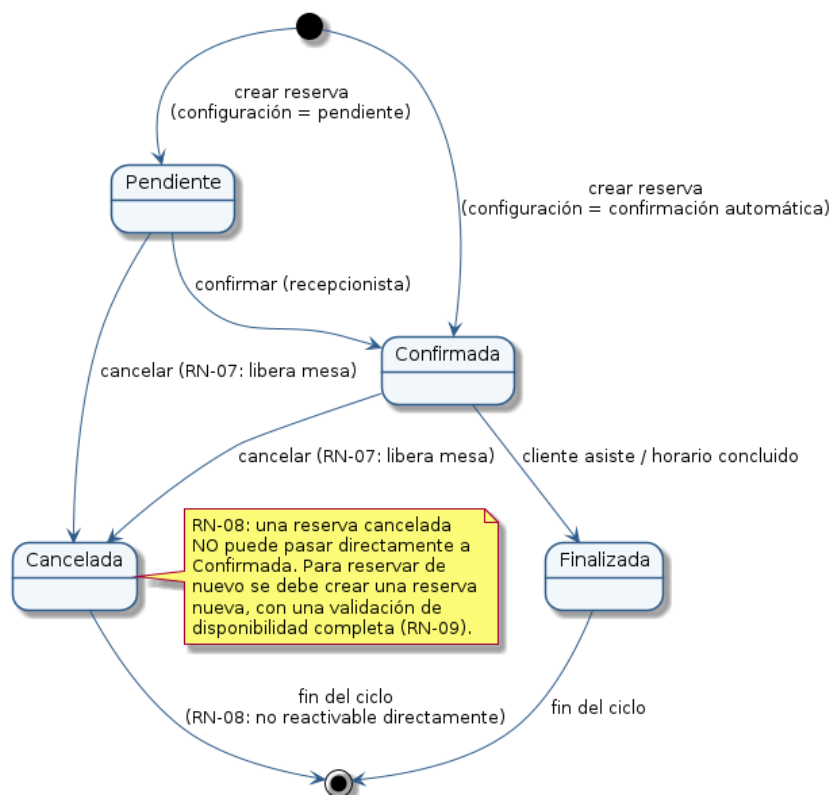


Figura 5. Diagrama de estados de la clase Reserva.

8.1 Descripción

Una reserva nace en estado Pendiente o Confirmada según la configuración del restaurante (RN-06, ConfiguracionRestaurante). Desde Pendiente puede pasar a Confirmada (acción del recepcionista) o a Cancelada. Desde Confirmada puede pasar a Cancelada o a Finalizada cuando concluye el horario de la visita.

La transición prohibida —y por eso resaltada explícitamente en el diagrama— es Cancelada → Confirmada de forma directa, en cumplimiento estricto de RN-08. Si el cliente desea reservar nuevamente después de cancelar, debe iniciarse un nuevo proceso de creación de reserva, pasando otra vez por la validación completa de disponibilidad (RN-09), tal como lo muestran el diagrama de secuencia y el de actividades.

9. Diagrama de Componentes

Se incluye este diagrama porque aporta valor al proyecto: permite ver, a nivel lógico, cómo se organizaría el sistema como aplicación web modular, lo cual facilita explicar el mantenimiento y la separación de responsabilidades exigidos por RNF-07.

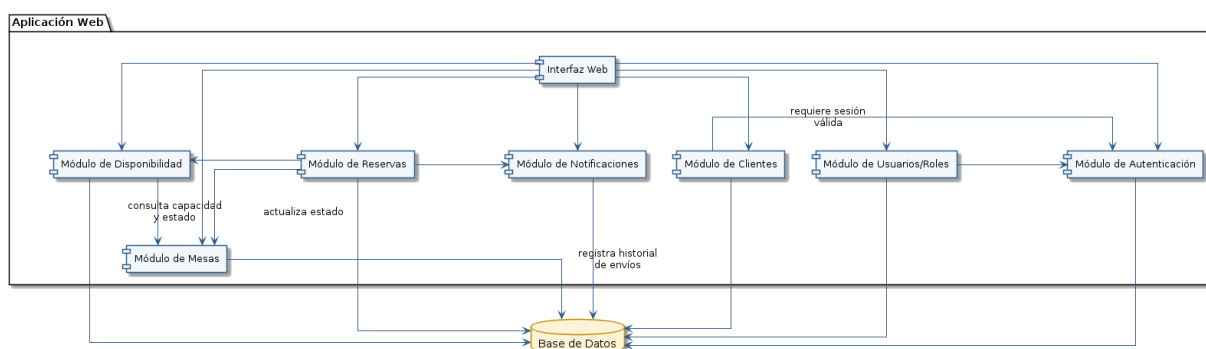


Figura 6. Diagrama de componentes del Sistema de Reservas de Restaurante.

9.1 Responsabilidades de cada componente

- Interfaz Web: presenta las pantallas a los tres actores y envía sus solicitudes a los módulos correspondientes.
- Módulo de Autenticación: valida credenciales, gestiona sesiones y aplica el control de acceso por rol (RF-15, RF-16, RNF-03).
- Módulo de Clientes: registra, busca y actualiza clientes (RF-01, RF-17, RF-19), y depende del Módulo de Autenticación para saber quién es el cliente que edita su propio perfil.
- Módulo de Mesas: registra y edita mesas, y mantiene su estado actualizado (RF-02).
- Módulo de Disponibilidad: implementa la consulta y la validación segura de cruces de horario (RF-03, RF-09, RF-10), y depende del Módulo de Mesas para conocer capacidad y estado.
- Módulo de Reservas: orquesta la creación, modificación y cancelación de reservas (RF-04 a RF-12), consultando al Módulo de Disponibilidad y solicitando notificaciones.

- Módulo de Usuarios/Roles: administra usuarios, roles y permisos (RF-13, RF-14) apoyándose en el Módulo de Autenticación.
- Módulo de Notificaciones: genera y envía las notificaciones de confirmación (RF-06).
- Base de Datos: persiste toda la información y garantiza, mediante transacciones, la integridad exigida por RNF-04.

10. Diagrama de Despliegue

Se incluye una versión básica del diagrama de despliegue porque el proyecto está definido como una aplicación web (RT-01) y este diagrama ayuda a visualizar, sin agregar infraestructura innecesaria, dónde se ejecuta cada parte del sistema.

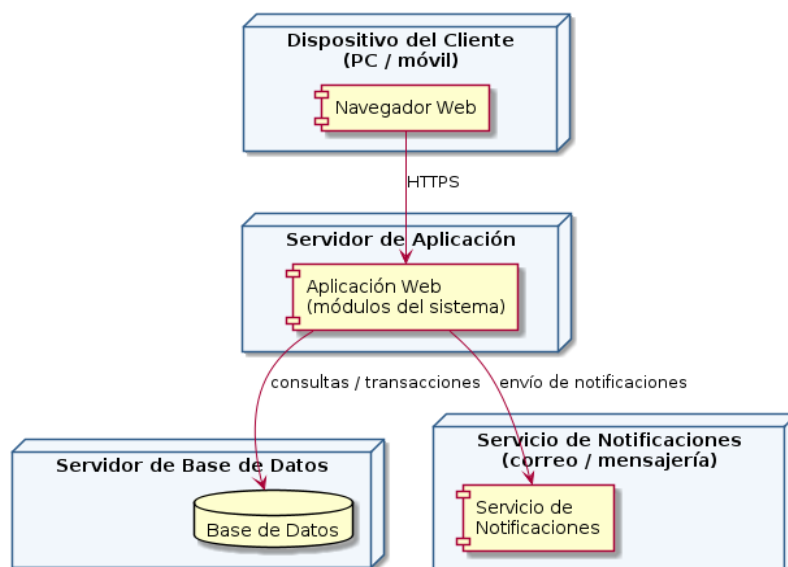


Figura 7. Diagrama de despliegue del Sistema de Reservas de Restaurante.

El dispositivo del cliente accede mediante un navegador web moderno (RNF-08) al servidor de aplicación, que aloja los módulos descritos en el diagrama de componentes. El servidor de aplicación se comunica con el servidor de base de datos para las operaciones de persistencia y con el servicio de notificaciones para el envío de confirmaciones (RF-06). No se agregan balanceadores, CDN u otra infraestructura, por no estar contemplados en el alcance ni en las restricciones del proyecto (RT-05).

11. Matriz de Trazabilidad UML

La siguiente matriz relaciona cada historia de usuario con su(s) requerimiento(s) funcional(es), el caso de uso correspondiente, la(s) clase(s) principal(es) involucrada(s) y el diagrama donde se representa.

HU	RF	Caso de uso	Clase(s) involucrada(s)	Diagrama(s)
HU-01	RF-01	Registrarse	Cliente, Usuario	Casos de uso, Clases
HU-02	RF-02	Gestionar mesas	Mesa, Recepcionista	Casos de uso, Clases
HU-03	RF-03	Consultar disponibilidad	Mesa, Reserva	Casos de uso, Clases, Secuencia, Actividades
HU-04	RF-04, RF-05, RF-06	Crear reserva	Reserva, Mesa, Cliente, Notificación	Todos los diagramas
HU-05	RF-07, RF-08	Consultar panel del día / Filtrar reservas	Reserva, Recepcionista	Casos de uso, Clases
HU-06	RF-09, RF-10	Validar disponibilidad	Reserva, Mesa	Casos de uso, Clases, Secuencia, Actividades

HU	RF	Caso de uso	Clase(s) involucrada(s)	Diagrama(s)
HU-07	RF-11, RF-12	Cancelar / Modificar reserva	Reserva, Mesa	Casos de uso, Clases, Estados
HU-08	RF-13, RF-14	Gestionar roles y permisos	Rol, Permiso, RegistroCambio, Administrador	Casos de uso, Clases
HU-09	RF-15, RF-16	Iniciar / Cerrar sesión	Usuario	Casos de uso, Clases
HU-10	RF-17	Actualizar perfil	Cliente	Casos de uso, Clases
HU-11	RF-18	Consultar historial	Reserva, Cliente	Casos de uso, Clases
HU-12	RF-19	Buscar cliente	Cliente, Recepcionista	Casos de uso, Clases
Sin HU/RF asociad o (ver nota)	Sin HU/RF asociad o (ver nota)	Configurar parámetros del restaurante	ConfiguracionRestaurant e, Administrador	Casos de uso, Clases

Nota: RF-20 (recuperación de acceso) se representa como una extensión del caso de uso Iniciar sesión, aunque no corresponde a una historia de usuario numerada de forma independiente en la Fase 1.

Nota: la fila “Configurar parámetros del restaurante” no tiene una HU ni una RF formal que la respalde (ver nota de alcance en las secciones 4.4 y 5.1); se incluye en la matriz para no dejarla fuera silenciosamente del rastreo de trazabilidad.

12. Conclusiones

El modelado UML desarrollado en esta Fase 3 confirma que las 12 historias de usuario y los 20 requerimientos funcionales definidos en las Fases 1 y 2 pueden representarse con una estructura de clases compacta (11 clases/enumeraciones), sin necesidad de funcionalidades fuera del alcance acordado.

La regla de negocio más crítica del proyecto —evitar que dos reservas activas se crucen para la misma mesa, fecha y horario (RN-01)— queda representada de forma consistente en los cuatro diagramas obligatorios: como una relación <<include>> en el diagrama de casos de uso, como el método validarDisponibilidad() en el diagrama de clases, como una doble verificación con transacción segura en el diagrama de secuencia, y como un nodo de decisión con retorno al flujo de consulta en el diagrama de actividades.

El análisis de consistencia realizado antes del modelado permitió detectar y resolver de forma explícita cuatro discrepancias entre las Fases 1 y 2 (dirección de una dependencia, dos re-estimaciones de puntos y dos conjuntos de dependencias incompletos), evitando que dichas contradicciones se trasladaran al modelo UML.

Finalmente, los diagramas complementarios de estados, componentes y despliegue, aunque no obligatorios, aportan valor porque documentan explícitamente una regla de negocio delicada (RN-08), la organización modular del sistema y su arquitectura básica de despliegue como aplicación web, sin incorporar infraestructura innecesaria.